

🩺 LLM-Code-Medic: 工业级多文件智能协同修复系统智能体系统设计方案

本设计方案全面对齐 LLM-Code-Medic 系统的实际生产架构。系统专为企业级复杂代码库的无监督自愈打造，底层依托 LangGraph 有向循环图拓扑、FastAPI 异步高并发网关、Pydantic v2 强类型契约，并独创性地采用“DeepSeek 核心诊断 + Gemini 专项修复”的异构双擎流控机制，实现零代码退化的自主修复闭环。

1. 系统架构图（System Architecture Diagram）

系统采用“前端轻量感知、网关强类型契约、异构双擎认知流控、沙箱物理隔离”的解耦拓扑架构。其核心控制反馈流向如下：

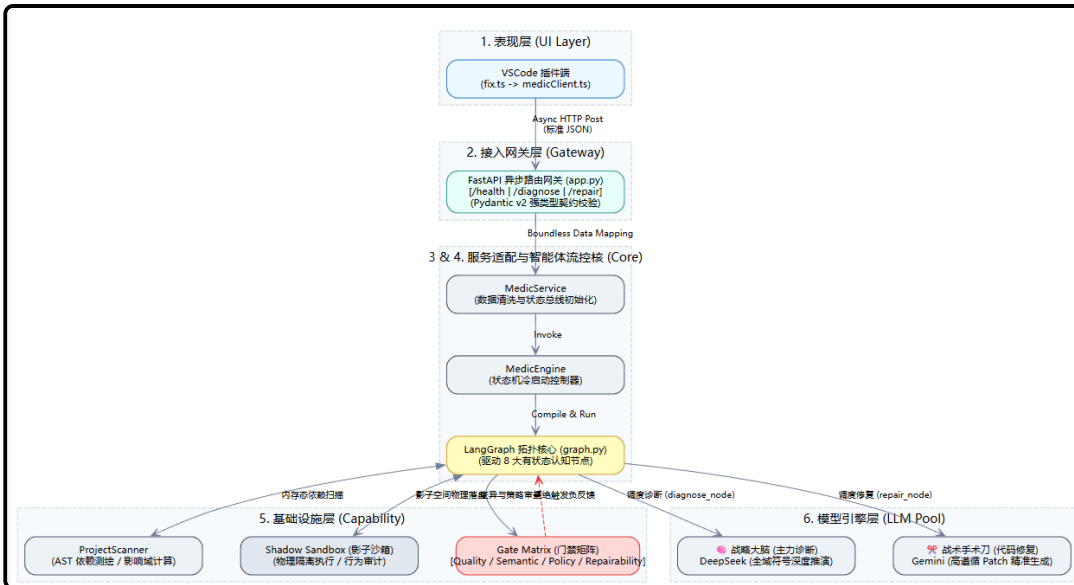


Figure 1

2. 全量架构与目录解析（Module Breakdown）

工程代码结构遵循高内聚、低耦合的现代云原生微服务与有状态智能体混合架构设计：



```

21 | | | | main.py # 传统 CLI 批量跑分测试终端本地启动入口
22 | | | | └─ config/ # 全局运行时配置解析与静态安全模式审查
23 | | | | |   └─ runtime_config.py # 统一加载环境变量、进行类型强转, 校验 API 密钥存活态
24 | | | | └─ service/ # ⚙️ 后端业务逻辑适配与网关桥接解耦层
25 | | | | |   └─ medic_service.py # [MedicService] 将网关层 HTTP 请求清洗转化为图状态总线字典
26 | | | | └─ engine/ # 🚚 核心业务流控编排引擎 (守护进程级)
27 | | | | |   └─ medic_engine.py # 状态机冷启动控制器, 拉起底层有向图并触发执行
28 | | | | |   └─ repair_pipeline.py # 修复流水线顶层管理器 (衔接拓扑扫描、图迭代与后置治理)
29 | | | | |   └─ runtime_session.py # 运行时单例会话管理器, 常驻维护底层多模型 HTTP 连接池
30 | | | | └─ agent/ # 🧠 认知智能体感知与核心拓扑流控层
31 | | | | |   └─ state.py # 全局 TypedDict [AgentState] 状态总线契约声明
32 | | | | |   └─ graph.py # 基于 LangGraph 的有向循环图控制核, 定义节点与条件决策路由
33 | | | | |   └─ prompts.py # 认知协议规约, 硬核内置思考路径与十维自查问卷约束
34 | | | | └─ llm/ # 🌐 弹性多供应商多模型高韧性容灾容错层
35 | | | | |   └─ model_factory.py # 统一大模型工厂, 标准化包装代理, 全面关闭底层 SDK 盲目重试
36 | | | | |   └─ provider_router.py # 运行时认知角色静态分流与降级控制中心
37 | | | | |   └─ llm_invoker.py # 弹性原子调用器, 植入指数退避重试算法 (Weighed Backoff)
38 | | | | └─ quality/ # 🔍 AST 静态代码语义与策略安全防护门禁矩阵 (编译前过滤层)
39 | | | | |   └─ patch_quality_gate.py # 补丁文件指纹与完整性门禁, 防止 AI 篡改文件或截断代码
40 | | | | |   └─ semantic_patch_gate.py # AST 语义红线审查器, 硬核挂载 Rule 0-7 变异探测机制
41 | | | | |   └─ policy_gate.py # 安全策略网关, 防止大模型恶意覆盖或篡改项目敏感业务常量
42 | | | | |   └─ repairability_gate.py # 收敛可修复性评估器, 死循环阻断及人机协同 Guided 交互放
权
43 | | | | └─ tools/ # 🛠️ 静态感知工具集与虚拟执行运行时
44 | | | | |   └─ scanner.py # [ProjectScanner] 核心雷达, 通过 ast.parse 测绘三层图谱
45 | | | | |   └─ ast_resolver.py # 符号依赖解析器, 基于图论实现影响域前向闭包无限迭代计算
46 | | | | |   └─ executor.py # [CodeExecutor] 影子沙箱空间, 提供安全写入、无状态跑测与行
为审计
47 | | | | |   └─ plugins/ # 🌸 后置代码高级治理插件总线 (责任链串行分发架构)
48 | | | | | |   └─ base_plugin.py # 行为契约抽象基类, 规范统一 run 接口
49 | | | | | |   └─ plugin_manager.py # 集中式链式串行调度引擎, 控制插件链条流转
50 | | | | | |   └─ ... # 包含代码美化、安全复审及反向单测自动补全插件
51 | | | | └─ output/ # 📁 Shadow Workspace 影子隔离沙箱物理执行区 (独立无状态磁
盘映像)
52 | | | | └─ tests/ # 🧪 复杂漏洞故障工程评估跑分标准基准库 (Benchmarks)

```

Fence 1

3. 3. 核心数据流 (Data Flow Lifecycle)

系统的数据流转严格区分内存态虚拟变换与磁盘态影子隔离执行, 大模型按节点分工精确调用。

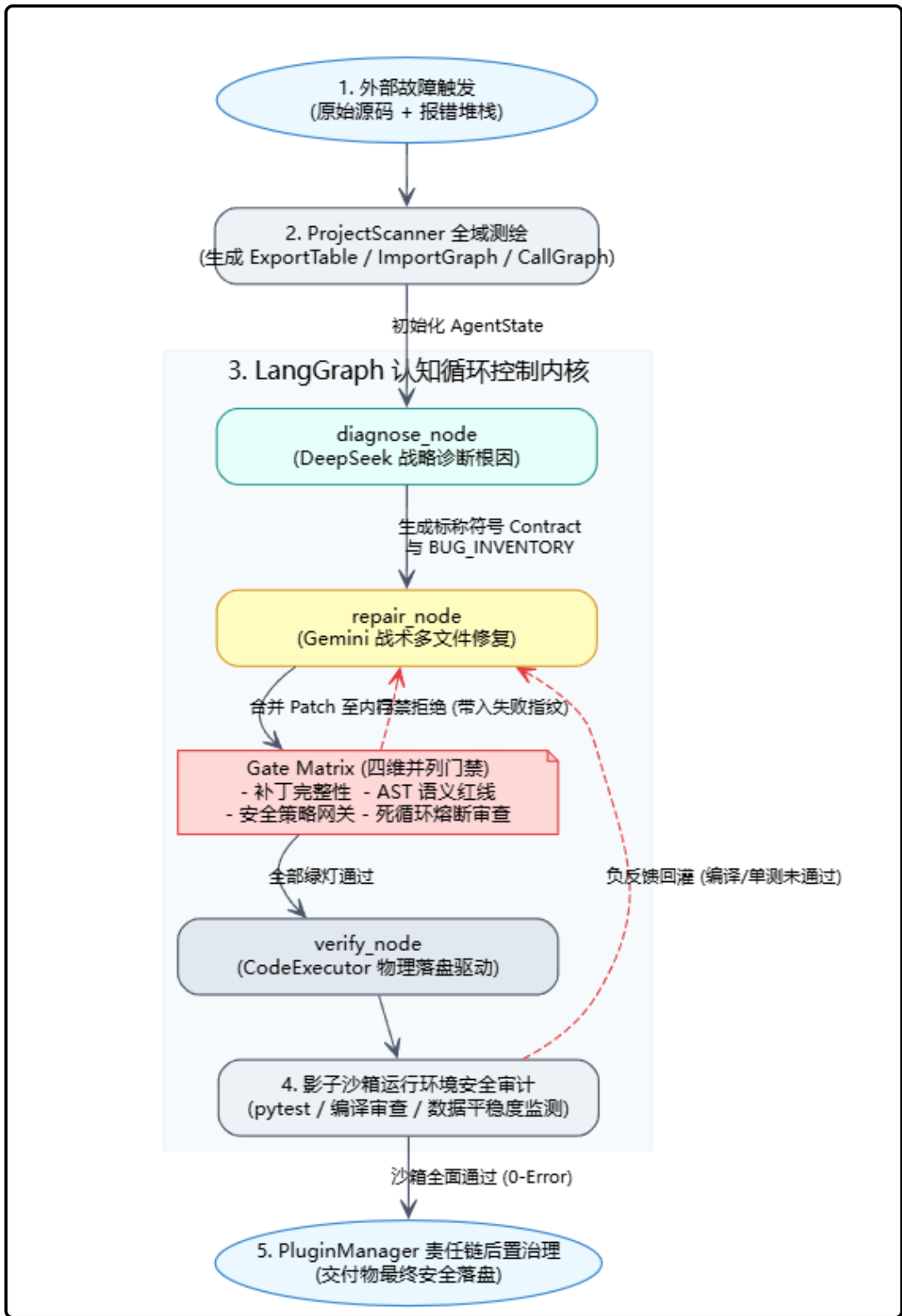


Figure 2

4. 4. 核心类与接口设计 (Core Class & Interface Design)

系统基于强类型契约构建，完美衔接 FastAPI DTO、LangGraph 状态字典与异构大模型调度。

4.1 4.1 网关强类型契约层 (server/schemas/)

```
1 | from pydantic import BaseModel, Field
2 | from typing import List, Dict
3 |
```

```

4 class RepairRequest(BaseModel):
5     """基于 Pydantic v2 的无状态修复请求接收契约 DTO"""
6     repo_root: str = Field(..., description="待修项目在宿主机的物理绝对路径")
7     error_message: str = Field(..., description="编译器抛出的原始报错堆栈或崩溃上下文")
8
9 class RepairResponse(BaseModel):
10    """基于 Pydantic v2 的端到端响应输出契约 DTO"""
11    success: bool = Field(..., description="HTTP请求及引擎调度是否成功执行")
12    is_fixed: bool = Field(..., description="沙箱及门禁最终跑测自愈判定状态")
13    repairable: bool = Field(..., description="系统是否判定该故障具备自动化修复可行性")
14    analysis: str = Field(..., description="DeepSeek 产出的深度根因分析与漏洞清单报告")
15    root_cause: str = Field(..., description="故障的静态分类归因")
16    modified_files: List[str] = Field(..., description="实际发生物理写入变动的文件相对路径清单")
17    verify_passed: bool = Field(..., description="影子沙箱自动化测试的跑测通行状态")
18    final_patch: str = Field(..., description="最终合入主干的原始补丁文本")

```

Fence 2

4.2 异构认知节点核心逻辑 (src/agent/graph.py)

```

1 def diagnose_node(state: AgentState) -> Dict[str, Any]:
2     """
3     【主力中枢：战略诊断节点】
4     调度核心主力模型 DeepSeek，吃入全域 AST 符号依赖关系，推演并锁定根因。
5     """
6     # 强制路由至 DeepSeek
7     provider = ProviderRouter.get_diagnose_provider() # 返回 "deepseek"
8     model_name = ProviderRouter.get_diagnose_model()
9
10    # 构建高密度全局上下文
11    ast_context = build_ast_context(state)
12    full_repo_snapshot = build_repo_snapshot(state["repo_files"])
13
14    user_prompt = f"请深度分析以下故障堆栈，并测绘全域符号 Contract，产出修复战略清单\nBUG_INVENTORY: \n{state['error_message']}\n\nAST Context:\n{ast_context}\n\nRepo\nSnapshot:\n{full_repo_snapshot}"
15
16    response = LLMInvoker.invoke(provider=provider, model_name=model_name, messages=[HumanMessage(content=user_prompt)], temperature=0.1)
17
18    return {**state, "analysis": response.content, "bug_inventory":\nextract_bug_inventory(response.content)}
19
20
21 def repair_node(state: AgentState) -> Dict[str, Any]:
22     """
23     【战术利刃：代码修复节点】
24     调度 Gemini 模型，严格基于 DeepSeek 的诊断报告 (analysis/bug_inventory) 产出精准的代码\nPatch 文本块。
25     """
26     # 强制路由至 Gemini
27     provider = ProviderRouter.get_repair_provider() # 返回 "gemini"
28     model_name = ProviderRouter.get_repair_model()
29
30    # 组装包含 DeepSeek 诊断结果与沙箱负反馈的战术提示词
31    user_prompt = f"请基于主力诊断报告修复目标文件。 \n核心诊断分\n析:\n{state.get('analysis')}\n\n错误清单:\n{state.get('bug_inventory')}\n\n最近沙箱报\n错:\n{state.get('sandbox_stderr')}"
32
33    response = LLMInvoker.invoke(provider=provider, model_name=model_name, messages=[SystemMessage(content=REPAIR_SYSTEM_PROMPT), HumanMessage(content=user_prompt)],\ntemperature=0.1)
34
35    # 解析补丁并合并入内存态虚拟文件字典
36    updates = parse_patch_response(response.content,\nexisting_keys=list(state["repo_files"].keys()))
37    merged_repo_files = dict(state["repo_files"])
38    merged_repo_files.update(updates)

```

```
return {**state, "repo_files": merged_repo_files, "repaired_repo_files":  
merged_repo_files, "final_patch": response.content, "last_patch_files":  
list(updates.keys()), "repair_attempts": state.get("repair_attempts", 0) + 1}
```

5. 关键技术选型理由（Tech Stack Validation）

系统的核心组件选型极具工业生产落地的说服力，通过完美的职能分工实现效能最大化：

本地异步网络接入网关：FastAPI

- 理由：基于原生 ASGI 异步的高并发性能，非常适合处理智能体多轮试错产生的长周期连接（Long Run HTTP 阻塞挂起请求）；能够完美无缝地与 VS Code 插件端的异步长轮询交互协议对接，保证连接不中断、状态不丢失。

强类型数据流契约层：Pydantic v2

- 理由：利用 Rust 重构后的 Pydantic v2 具备极高的解析性能。在网关层作为 DTO 执行严格的代码级类型提示与多字段强校验，彻底过滤不合规的非法请求；同时对智能体最终输出的复杂 JSON 字典进行严格边界对齐，防止前后端字段踩踏。

有状态图控编排引擎：LangGraph

- 理由：代码自愈本质上是个高频发生“修复-因代码变异被门禁拦截-重试-编译通过但单测报错-重新诊断”的有向循环图（DAG with Cycles）。LangGraph 提供了原生有状态节点管理与灵活的条件边判断（`add_conditional_edges`），是构建此类自反思（Self-Reflection）反馈控制流的核心解。

主力核心战略诊断引擎：DeepSeek

- 理由：作为系统的“主力中枢与诊断大脑”。DeepSeek 在跨模块全局符号链条的长文本推演（Reasoning）和根因定位上表现出惊人的准确度。通过让其专项负责 `diagnose_node`，能够深度榨干其长链条逻辑思维红利，产出高质量、高内聚的 `BUG_INVENTORY`，为后续修复画好精准的牢笼边界。

专项战术代码生成利刃：Gemini

- 理由：作为系统的“修复手术刀”。Gemini 拥有极其强悍的代码上下文感知、超长的原生上下文窗口（Context Window）以及出色的战术级指令遵循能力。在已知 DeepSeek 圈定的核心故障清单前提下，Gemini 负责 `repair_node` 能够极速稳定地输出高度对齐工作区目录结构的 Patch 块，出片规整、极少截断，与 DeepSeek 形成了完美的“战略+战术”互补选型。